LLM SCA How far are we?

Abstract

1 Motivation

The proliferation of Large Language Models (LLMs) has been dramatically accelerated by collaborative platforms like the Hugging Face Hub, which serves as a central repository for model sharing. A cornerstone of this ecosystem is the iterative development of new models by building upon existing ones through techniques such as fine-tuning, merging, and quantization. This process creates a complex web of inter-model dependencies, which we define as **model genealogy** or **lineage**. A clear understanding of a model's genealogy is paramount for ensuring traceability, reproducibility, and compliance, as derivative models often inherit biases, vulnerabilities, or licensing constraints from their predecessors.

However, the mechanisms currently available on the Hugging Face Hub for documenting this critical lineage information are frequently inconsistent, incomplete, or entirely absent. This lack of a systematic and enforceable standard for provenance documentation poses a significant barrier to researchers and developers. The most visible symptom of this issue is the difficulty, or outright impossibility, of programmatically constructing a model's lineage–a structure ideally represented as a "Model Tree". This is not a superficial interface problem but a direct consequence of fundamental flaws in how provenance data is recorded at its source.

To illustrate the severity of this challenge, we will dissect two distinct modes of documentation failure, each representing a progressively worsening degree of information obfuscation.

1.1 Problem 1: Lineage Concealed in Unstructured Text

In the most common scenario, lineage information exists but is locked within free-form text, rendering it inaccessible to automated tools. This forces researchers into a manual, error-prone process of discovery.

Case Study 1: `DavidAU/NSFW_DPO_Noromaid-7b-GGUF`

The Hugging Face repository of this model does not include a model tree; however, its model card explicitly states:

> *NSFW_DPO_Noromaid-7b-Mistral-7B-Instruct-v0.1 is a merge of the following models: mistralai/Mistral-7B-Instruct-v0.1 and athirdpath/NSFW_DPO_Noromaid-7b.*

Notably, the Hugging Face pages of both referenced models in the model card contain their respective model trees. This reliance on unstructured documentation makes the automated construction of a lineage graph infeasible. It imposes a significant manual burden for identifying even a direct parent and completely obstructs any large-scale, systematic analysis of the model ecosystem.

Case Study 2: `timdettmers/guanaco-13b`

Another example is the `timdettmers/guanaco-13b` model. While this model is a fine-tuned version of LLaMA, this pivotal relationship is not captured in any structured, machine-readable metadata. To discover this link, a user must manually navigate to the model's repository, open the "Model Card" (a Markdown file), and parse through natural language prose to find the statement:

> *Guanaco models are open-source finetuned chatbots obtained through 4-bit QLoRA tuning of LLaMA base models...*

1.2 Problem 2: Lineage Outsourced to Fragile External Links

A more precarious situation arises when provenance information is not even located within the Hugging Face platform, but is instead hyperlinked to an external, third-party source.

Case Study 3: `pfnet/plamo-2.1-2b-cpt`

The model's homepage lacks a model tree, and its model card merely states that the model was derived by reducing the parameter size from an 8B-parameter model, without explicitly specifying the base model used for this operation. However, the model card provides links to three technical blogs for reference. By following these external links, we can infer that the mentioned 8B-parameter model is `plano-2-8b`.

Case Study 4: `Remilistrasza/Mysterious_SDXL_Lah`

Consider the `Remilistrasza/Mysterious_SDXL_Lah` model. Its Model Card is starkly empty of descriptive content, offering only a single hyperlink to a page on Civitai, another model-sharing platform. To identify the base model, a user is forced to leave the Hugging Face ecosystem. Only upon navigating to this external page does one discover that the base model is "*SDXL 1.0*".

This practice introduces two severe risks: **information fragmentation** and **link rot**. The integrity and availability of the lineage information become wholly dependent on the uptime and archival policies of an external website. Should the link become invalid (a 404 error), or the content on the remote page be altered or deleted, the model's traceable lineage is permanently severed. This practice renders the model's provenance fragile and unverifiable from within the Hugging Face ecosystem.

These documented failures, which range from information concealed in prose to its complete omission, demonstrate a clear and pressing need for an automated and reliable method to construct

model genealogies. The current reliance on inconsistent and manual documentation is an unsustainable bottleneck in an ecosystem that is scaling rapidly. To bridge this critical gap, we propose an approach to automatically identify and verify the derivative relationships between models, thereby enabling robust and scalable lineage tracing.

2 Benchmark

2.1 Construction

The construction of our benchmark involves a systematic pipeline that traces model derivations from base models to their descendants on Hugging Face. This process is divided into five key steps:

Step 1: Obtaining Base Model List. We begin by collecting a list of popular base models for the `text-generation` task from Hugging Face. Utilizing the `HfApi` provided by `huggingface_hub`, we retrieve the top 50 non-gated models sorted by the number of likes. The model identifiers are stored in a file named `base_models.txt`, where each line corresponds to one base model ID. This step is implemented via **Script 1**.

Step 2: Building the Model Derivation Tree. Based on `base_models.txt`, we recursively crawl derivative models to construct a "model forest". For each base model, we explore its derivatives up to four levels deep, covering four derivation types: `finetune`, `adapter`, `quantized`, and `merge`. For each derivation type, a maximum of six sibling nodes are considered. Every node in the tree records metadata such as model name, crawl timestamp, depth level, and its child nodes. The resulting forest structure is serialized into `model_forest.json`. This step is implemented via **Script 2**.

Step 3: Constructing Model Pairs. From the model forest, we extract two categories of model pairs: adjacent parent-child pairs and non-adjacent ancestor-descendant pairs. By performing a Breadth-First Search (BFS) traversal on each tree, we generate:

- **Adjacent Pairs**: Direct parent-child model pairs.
- **Non-Adjacent Pairs**: All ancestor-descendant pairs where the ancestor is not the immediate parent.

These pairs are recorded in `adjacent_pairs.jsonl` and `non_adjacent_pairs.jsonl`, respectively. This step is implemented via **Script 3**.

Step 4: Single-Threaded Model Validation. To ensure data quality, we validate the structural correctness of all adjacent model pairs. For each model, we inspect its file tree on Hugging Face, checking for the presence of critical files such as `config.json`, `adapter_config.json`, and various parameter files (e.g., `.bin`, `.pt`, `.safetensors`, etc.). We verify whether the claimed derivation type aligns with the actual file structure and whether the parent model is valid. The results are compiled into a comprehensive validation report named `model_validation_report.json`. This step is implemented via **Script 4**.

Step 5: Filtering Erroneous Model Pairs. Based on the validation report, we remove all erroneous model pairs from the dataset. Models flagged with any inconsistencies or missing critical files are excluded. The cleaned set of adjacent model pairs is saved in `adjacent_pairs_without_error.json`. This step is implemented via **Script 5**.

Table 1: Statistics of the GrandMoTHer benchmark.

Statistic	Value
#Graphs	31
#Model pairs (edges)	248
Min / Median / Max pairs per graph	1 / 6 / 22
Avg. pairs per graph	8.0
#Models (nodes)	279
#Root models	31
#Intermediate models	44
#Leaf models	204
Share of leaf models	73.1%
Min / Avg / Max models per graph	2 / 9.0 / 23
Avg. out-degree of non-leaf models	3.31
Max fine-tuning depth	2

2.2 Characteristics

Our benchmark exhibits several notable characteristics that reflect its diversity and representativeness of the Hugging Face model ecosystem:

- **Variety of Derivation Types**: The dataset covers four major derivation types—`finetune`, `adapter`, `quantized`, and `merge`—capturing a wide range of model transformation patterns.
- **Hierarchical Depth**: With a recursive crawling depth of up to four levels, the benchmark contains both shallow and deep derivation chains, enabling analysis of both immediate and long-range model inheritance.
- **Scale and Coverage**: Starting from 50 popular base models, the recursive traversal results in a substantial model forest, ensuring coverage of highly influential models as well as their derivative variants.
- **Data Cleanliness**: Through rigorous validation and error filtering, the dataset ensures high structural integrity, making it suitable for tasks that require reliable model lineage information.
- **Real-World Relevance**: The benchmark reflects actual user-driven derivation patterns on Hugging Face, providing insights into community practices such as model fine-tuning trends, adapter usage, and quantization adoption.

3 Empirical Results

We design experiments to explore the following research questions.

- **Measurement Evaluation (RQ1)**: How do different measurement metrics affect the effectiveness of LLM lineage detection?
- **Layers in Transformer Block Evaluation (RQ2)**: How do different model layers influence the effectiveness of LLM lineage detection?
- **Failure Analysis (RQ3)**: Why do existing methods fail to detect lineage?
- **Integration (RQ4)**: using the above findings, the best performance can achieve? (80%)

Table 2: Lineage Detection Accuracy (%) w.r.t. Different Measurement Metrics, Direction Methods, and Graph Algorithms (RQ1)

Graph Algorithm	Dir.	Measurement Metric					
		L2	Cos.	AM	AMx	SA	SR
+ MoTHer	K	...	...	...	...	...	...
	T	...	...	...	...	...	...
+ Hard-Constraint	K	...	...	...	...	...	...
	T	...	...	...	...	...	...
+ Undirected-Mst	K	...	...	...	...	...	...
	T	...	...	...	...	...	...
+ atlas_snake_fan	K	...	...	...	...	...	...
	T	...	...	...	...	...	...

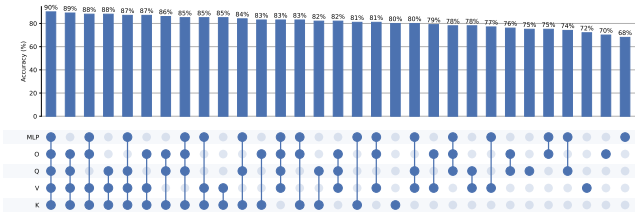


Figure 1: Accuracy of lineage detection with different combinations of transformer block components (RQ2).

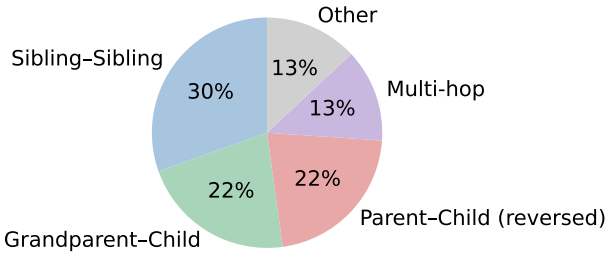


Figure 2: Proportion of different types of erroneous edges in the constructed lineage graph (RQ3).

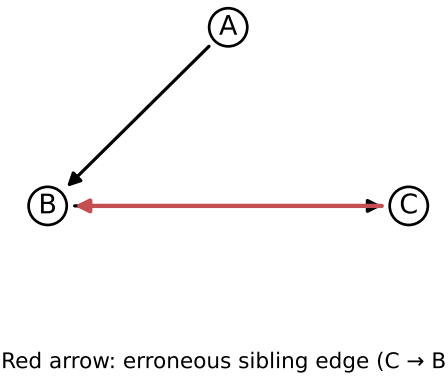


Figure 3: Case study illustrating erroneous edges in the constructed lineage graph (RQ3).

3.1 Setup

Measurement Metrics. XXX

Accuracy.

Layers. XXX

Graph Algorithms. XX

3.2 RQ1: Measurement Evaluation

Summary: L2+timestamp+xxgraph algorithm performs best.

3.3 RQ2: Layers in Transformer Block Evaluation

Summary: using QKV layer performs best. K layer contributes most.

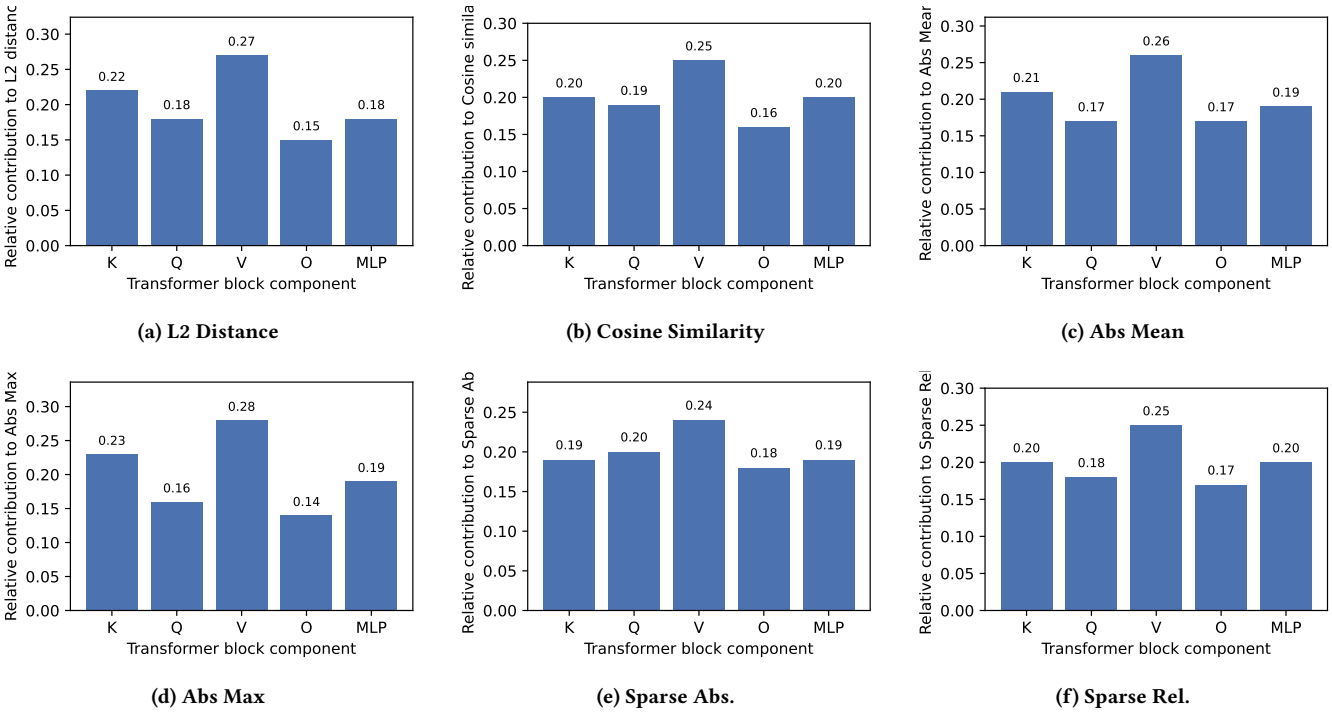
Summary:

3.4 RQ3: Failure Analysis

Multi-hop Analysis (RQ3): XX

4 Related Work

Model Attribution and Similarity Metrics. Determining the provenance of Large Language Models (LLMs) traditionally relies on watermarking, which identifies ownership from outputs but sacrifices generation quality [1, 7, 9, 20]. Alternatively, active model fingerprinting injects specific triggers via lightweight tuning [13, 14, 19]. However, these intrusive modifications risk performance degradation and exhibit poor robustness against complex fine-tuning [10]. Consequently, recent research has shifted toward post-hoc similarity metrics, which evaluate model relationships without prior injection. These approaches are broadly categorized into dynamic inference and static analysis. Dynamic methods evaluate pairwise similarity during runtime, operating either as black-box approaches comparing output distributions [11] or white-box approaches computing activation similarities (e.g., CKA) [23] and gradient statistics [18]. However, dynamic inference necessitates extensive computational resources and forward-pass overhead. This significant cost has driven a transition toward static analysis, which directly examines white-box weights. While initial static approaches utilize cross-layer statistical consistencies [6] or Singular Value Decomposition (SVD) [16], they can be vulnerable to weight manipulations. To robustly determine whether two models are related even under adversarial permutation and scaling attacks,



(a) L2 Distance **(b) Cosine Similarity** **(c) Abs Mean**

(d) Abs Max **(e) Sparse Abs.** **(f) Sparse Rel.**

Figure 4: Contribution of different transformer block components (K/Q/V/O/MLP) to various distance metrics (RQ2).

state-of-the-art static methods focus on computing structural invariants [22], often integrating SVD to extract these robust architectural signatures [17]. Despite their efficacy, all aforementioned metrics evaluate similarity strictly in a pairwise manner.

Ecosystem-Scale Model Lineage Reconstruction. Beyond pairwise verification, pioneering works scale similarity metrics to reconstruct macroscopic evolutionary graphs of the open-source ecosystem, aiming to uncover complex N-to-N genealogical trees [12]. Initial black-box methods leverage dynamic inference to compute vocabulary probability differences, such as Nei's genetic distance, for phylogenetic tree construction [21]. However, relying purely on distance yields unrooted trees that fail to infer evolutionary directionality and often misidentify root models. To resolve this lack of directionality, recent white-box approaches incorporate intrinsic or extrinsic signals. Specifically, they utilize weight-based L_2 distances combined with layer kurtosis [4] or metadata timestamps [3] to construct directed lineages, accurately charting how foundation models adapt and branch over time.

Layer-wise Dynamics in Transformers. Understanding the layer-wise internal mechanisms of Transformers has long been a focal point for model interpretability and architecture optimization. While early studies analyzed the hierarchical progression of linguistic features across hidden layers [8, 15], recent evidence suggests that Transformer components exhibit significant functional specialization and asymmetric sensitivity during model evolution. For instance, Feed-Forward Networks (FFN) are increasingly recognized as key-value memories that store factual knowledge [2], while attention sub-layers (Q, K, V) govern contextual alignment. The success of parameter-efficient fine-tuning (PEFT) further underscores this heterogeneity, demonstrating that updates to specific matrices like W_q and W_v are often sufficient to capture the majority of model shifts [5]. In the context of model provenance, specific weight blocks have been found to maintain invariant spectral signatures during reuse [16], yet existing lineage detection frameworks largely treat all parameters as a homogeneous vector space when calculating inter-model distances. It remains unexplored how individual components–namely the Q, K, V, and MLP matrices–differentially contribute to the fidelity and robustness of macroscopic lineage graph reconstruction.

References

[1] Miranda Christ, Sam Gunn, and Or Zamir. 2023. Undetectable Watermarks for Language Models. arXiv:2306.09194 [cs.CR] https://arxiv.org/abs/2306.09194

[2] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 5484–5495. https://doi.org/10.18653/v1/2021.emnlp-main.446

[3] Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. 2025. Charting and Navigating Hugging Face's Model Atlas. *arXiv preprint arXiv:2503.10633* (2025).

[4] Eliahu Horwitz, Asaf Shul, and Yedid Hoshen. 2025. Unsupervised Model Tree Heritage Recovery. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=QVj3kUvdvl

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL] https://arxiv.org/abs/2106.09685

[6] Do hyeon Yoon, Minsoo Chun, Thomas Allen, Hans Müller, Min Wang, and Rajesh Sharma. 2025. Intrinsic Fingerprint of LLMs: Continue Training is NOT All You Need to Steal A Model! arXiv:2507.03014 [cs.CR] https://arxiv.org/abs/2507.03014

[7] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2024. A Watermark for Large Language Models. arXiv:2301.10226 [cs.LG] https://arxiv.org/abs/2301.10226

[8] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. 2019. Similarity of Neural Network Representations Revisited. arXiv:1905.00414 [cs.LG]

https://arxiv.org/abs/1905.00414

[9] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. 2024. Robust Distortion-free Watermarks for Language Models. arXiv:2307.15593 [cs.LG] https://arxiv.org/abs/2307.15593

[10] Anshul Nasery, Jonathan Hayase, Creston Brooks, Peiyao Sheng, Himanshu Tyagi, Pramod Viswanath, and Sewoong Oh. 2025. Scalable Fingerprinting of Large Language Models. arXiv:2502.07760 [cs.CR] https://arxiv.org/abs/2502.07760

[11] Dario Pasquini, Evgenios M. Kornaropoulos, and Giuseppe Ateniese. 2025. LLMmap: Fingerprinting For Large Language Models. In *34th USENIX Security Symposium (USENIX Security 25)*.

[12] Mohammad Shahedur Rahman, Peng Gao, and Yuede Ji. 2025. HuggingGraph: Understanding the Supply Chain of LLM Ecosystem. arXiv:2507.14240 [cs.CL] https://arxiv.org/abs/2507.14240

[13] Mark Russinovich and Ahmed Salem. 2025. Hey, That's My Model! Introducing Chain & Hash, An LLM Fingerprinting Technique. arXiv:2407.10887 [cs.CR] https://arxiv.org/abs/2407.10887

[14] Huan Teng, Yuhui Quan, Chengyu Wang, Jun Huang, and Hui Ji. 2025. Fingerprinting Denoising Diffusion Probabilistic Models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 28811–28820. https://doi.org/10.1109/CVPR52734.2025.02683

[15] Betty van Aken, Benjamin Winter, Alexander Löser, and Felix A. Gers. 2019. How Does BERT Answer Questions? *Proceedings of the 28th ACM International Conference on Information and Knowledge Management - CIKM '19* (2019).

[16] Suqing Wang, Ziyang Ma, Li Xinyi, and Zuchao Li. 2025. Ghost in the Transformer: Detecting Model Reuse with Invariant Spectral Signatures. arXiv:2511.06390 [cs.CR] https://arxiv.org/abs/2511.06390

[17] Zhaomin Wu, Haodong Zhao, Ziyang Wang, Jizhou Guo, Qian Wang, and Bingsheng He. 2026. LLM DNA: Tracing Model Evolution via Functional Representations. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/pdf?id=UIxHaAqFqQ

[18] Zehao Wu, Yanjie Zhao, and Haoyu Wang. 2025. Gradient-Based Model Fingerprinting for LLM Similarity Detection and Family Classification. arXiv:2506.01631 [cs.LG] https://arxiv.org/abs/2506.01631

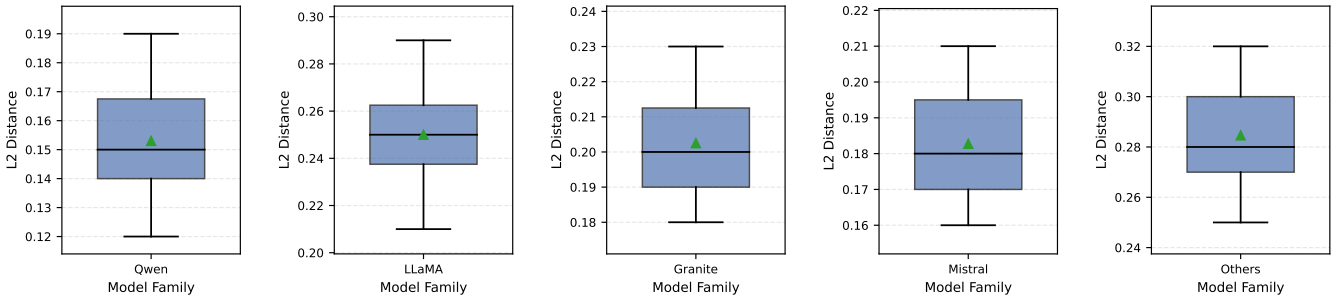
[19] Jiashu Xu, Fei Wang, Mingyu Ma, Pang Wei Koh, Chaowei Xiao, and Muhao Chen. 2024. Instructional Fingerprinting of Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 3277–3306. https://doi.org/10.18653/v1/2024.naacl-long.180

[20] Xi Yang, Kejiang Chen, Weiming Zhang, Chang Liu, Yuang Qi, Jie Zhang, Han Fang, and Nenghai Yu. 2023. Watermarking Text Generated by Black-Box Language Models. arXiv:2305.08883 [cs.CL] https://arxiv.org/abs/2305.08883

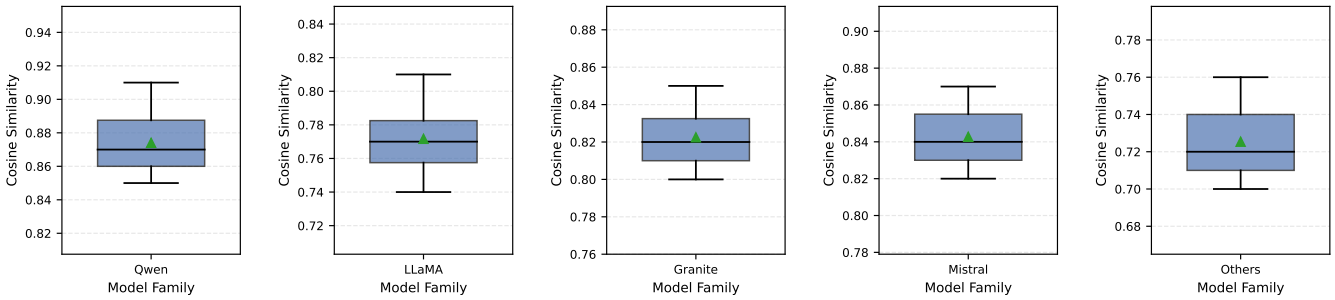
[21] Nicolas Yax, Pierre-Yves Oudeyer, and Stefano Palminteri. 2025. PhyloLM : Inferring the Phylogeny of Large Language Models and Predicting their Performances in Benchmarks. arXiv:2404.04671 [cs.CL] https://arxiv.org/abs/2404.04671

[22] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. 2024. HuRef: HUman-REadable Fingerprint for Large Language Models. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 126332–126362. https://doi.org/10.52202/079017-4013

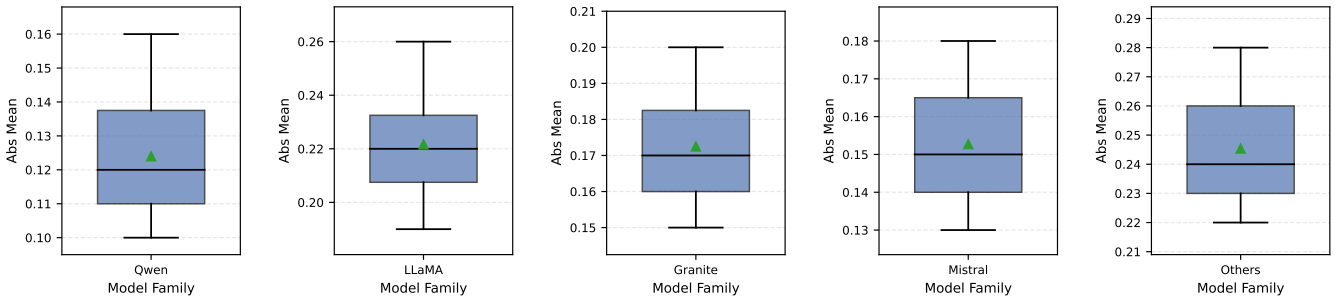
[23] Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. 2024. REEF: Representation Encoding Fingerprints for Large Language Models. arXiv:2410.14273 [cs.CL] https://arxiv.org/abs/2410.14273



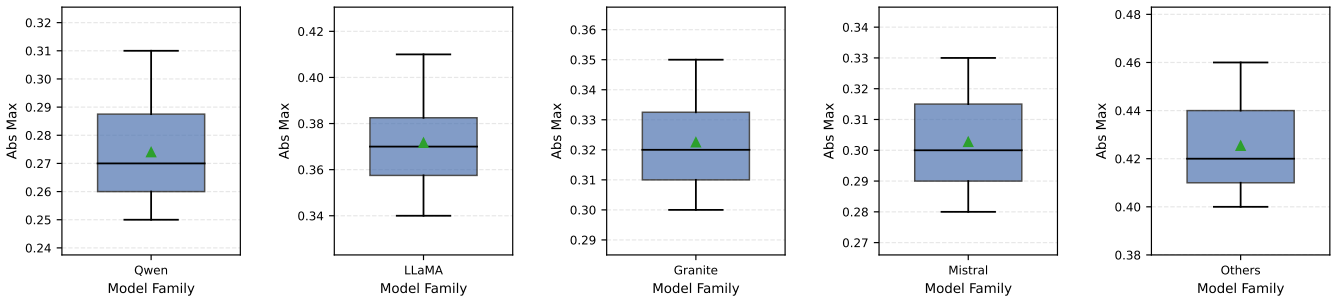
(a) L2 Distance



(b) Cosine Similarity



(c) Abs Mean



(d) Abs Max